
FX trading using Reinforcement Learning

Ivan Kolesnikov, 2026

Table of contents

01

Research problem

And literature review

02

Description of the model

Data sources, features, rewards

03

Empirical part

Training and validation results, and runs on synthetic data

04

Conclusion

Project limitations, future research and Q&A

01

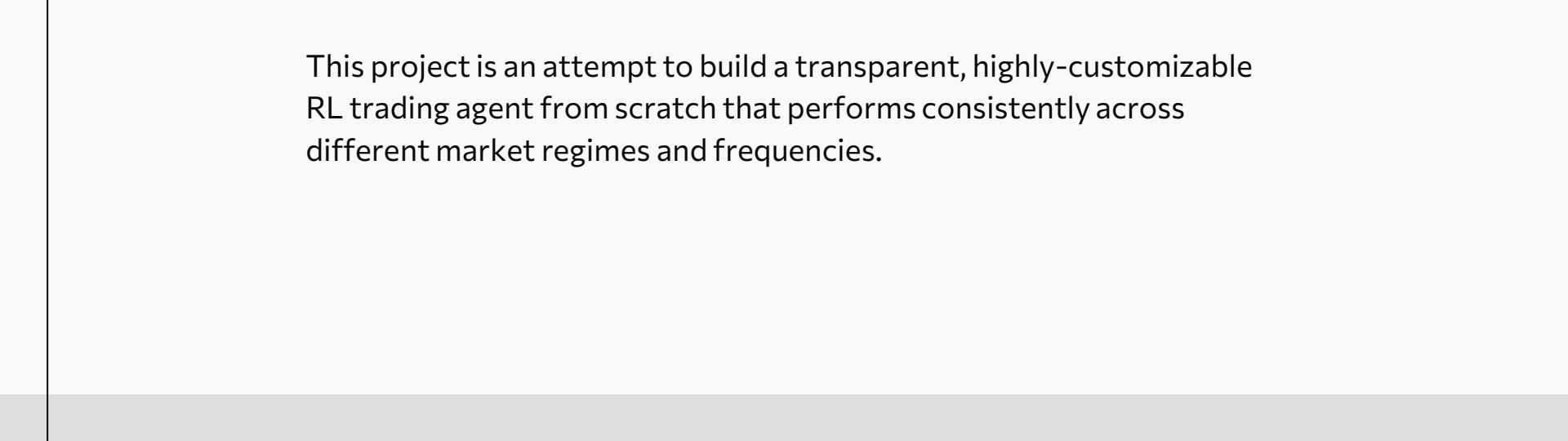
Research problem

And literature review

Research problem

Developing a robust FX trading strategy is challenging due to market noise, non-stationary data, and the risk of overfitting. Existing frameworks like FinRL are often black boxes for users.

This project is an attempt to build a transparent, highly-customizable RL trading agent from scratch that performs consistently across different market regimes and frequencies.

A thin vertical line is positioned on the left side of the slide, and a horizontal grey bar spans the bottom.

Description

This project implements a custom Gymnasium environment for foreign exchange (FX) trading (high-, mid-, low-frequency).

It includes data preprocessing, feature engineering, and a training pipeline using Ray RLlib

The environment and model are highly-customizable and were built from scratch, without relying on existing frameworks such as FinRL and TradeMaster

[GitHub link](#)

02

Description of the model

Data sources, features, rewards

Data sources

We use historical one-minute data from [philipperemy/FX-1-Minute-Data](#):

1. Data is provided in the OHLC format
2. It was loaded for 5 currency pairs: EURJPY, EURUSD, SGDJPY, USDJPY, USDSGD
3. It is available for 5 years (2020-2024)
4. All timestamps are in NYC time

We keep only main session trading hours (9-19) to avoid low liquidity and to imitate the real-life fund. Weekends are also excluded. Only real currency pairs are traded to avoid using synthetic rates.

If the data for timestamp is not available for one of the currencies it is assumed the price stayed the same.

Features

The model combines several groups of parameters:

1. Technical indicators (e.g. EMA, ADX, RSI) [Lu, L. (2025)]
2. Historical returns and volatility (from 1-minute to 2-hour trailing windows)
3. Time parameters (hour, minute, day of the week)
4. Current portfolio and market exchange rates

We also create reverse exchange rates: $\text{JPYEUR} = 1 / \text{EURJPY}$ to simplify computations later

Features preprocessing

We split the data into 3 groups:

1. 01.01.2020 – 31.05.2023 -> used to train NormalScaler and PCA
2. 01.06.2023 – 31.05.2024 -> used to train the RL algorithm
3. 01.06.2024 – 31.12.2024 -> validation set

We apply normalization and PCA to the first two groups (tech. indicators and hist. data):

1. We keep only eigen-vectors that explain more than 0.5% of the variance. It is slightly more conservative than Kaiser criterion (lowest eigen-value is 0.91 instead of 1.0)
2. The number of features in these groups decreases from 180 to 27
3. The total explained variance ratio is 94.5%

There are 35 features from all groups in the end.

Model action space

For each of the 5 currencies we predict flow in range $[-1, +1]$. Using a portfolio containing multiple currencies, e.g. {USD, EUR, JPY}, an agent should be able to purchase JPY using both USD and EUR simultaneously – a capability that direct weight optimization would miss. Modelling flows per currency pair implicitly allows such multi-source trades

For example, for a given currency pair XXXYYY (e.g., "EURUSD"):

- Positive action $\rightarrow$ buy the base currency (XXX) using the quote currency (YYY).
- Negative action $\rightarrow$ sell the base currency (XXX) for the quote currency (YYY).

Action is also multiplied by a set constraint $(0, 1]$ to ensure stability and better risk-management. 0.99 is used now.

Single trade

Let's assume that we have action A at time t for currencies X and Y and it has a positive sign (sell X, buy Y). Mathematically speaking, it looks the following way:

$$\begin{aligned}\Delta Q_x &= Q_{x,t-1} A_t W \\ \Delta Q_y &= \Delta Q_x * FX_{xy,t} * \max(1 - fee - |slippage|, 0)\end{aligned}$$

Where Q – respective quantity, A – action, W – weight scaler, FX – current exchange rate, fee – constant trading commission (1 bps), slippage – stochastic slippage $N(0.0001, 0.0002^2)$

Algorithm:

1. Ensures that the model does not sell more currency that it currently has in the portfolio by doing all trades in chain.
 2. Assumes the market is liquid enough for our volume and disregards further changes in trading book structure (due to the lack of data)
-

Reward

Total profit is used as a training reward. Differential Sharpe is also available, but it has proved to be a harder metric to learn as mentioned in [source]. As mentioned before, reward also contains trading fee, stochastic slippage and penalties for action and non-trading.

After completing all trades reward is computed as:

$$r_t = \ln(P_t) - \ln(P_0) - \xi \frac{\sum_{i=1}^n A_{i,t}^2}{n}$$

Where $P(t)$ – current portfolio value in base currency, $P(0)$ – initial portfolio value, ξ - action Penalty (0.05), A – actions.

- If $A_i < 0.0001 \forall i$, the penalty for non-trading is subtracted: $\tilde{r}_t = r_t - 0.5$
 - If $P(t) \leq 0.01$, the episode ends
-

Model structure

We use Ray RLlib Proximal Policy Optimization (PPO) algorithm with 28 simultaneous workers.

The model is trained for 200 episodes (16384 timesteps each). After a 100th episode, the early stopping condition is applied if an average reward improves less than 0.01 for 20 episodes.

1 worker's episode is either 1 day (600 timesteps) or 2 weeks (6000 timesteps). Both types' results are compared in the later part.

Model initialization (exploration part) is done via StochasticSampling with 5000 timesteps.

Decision-making part

The custom PyTorch model consists of the following parts:

Main network – three hidden layers ($256 \rightarrow 128 \rightarrow 64$), each combined with LayerNorm, GELU activation, and dropout (0.2). It is followed by:

1. **Mean network** – linear layer projecting the 64-dim features to the action dimension, followed by Tanh to bound actions in $[-1, 1]$.
2. **Log-std network** – linear layer producing the log-standard deviations for the action distribution (required by RLLib's StochasticSampling). The values are clamped to $[-3, 1]$
3. **Value network** – two layers ($64 \rightarrow 32 \rightarrow 1$) with GELU activation.

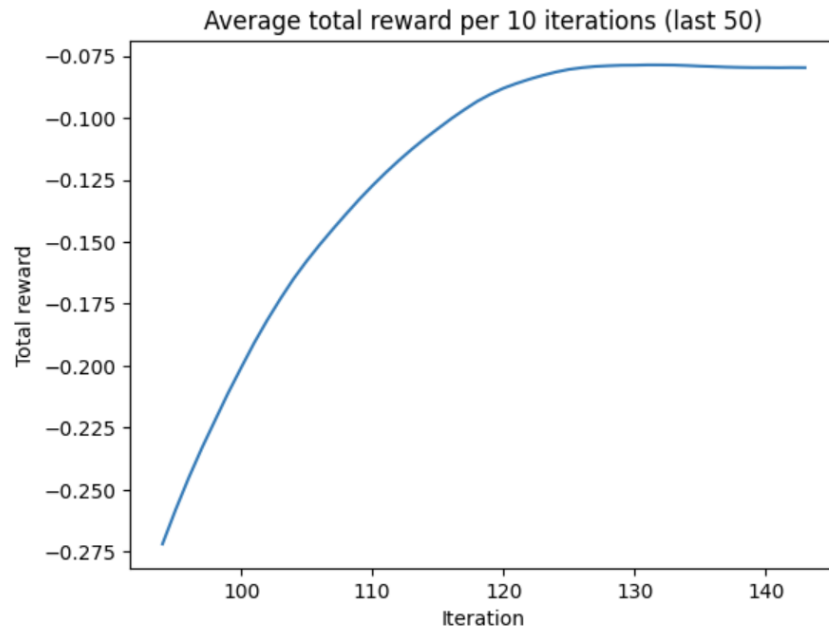
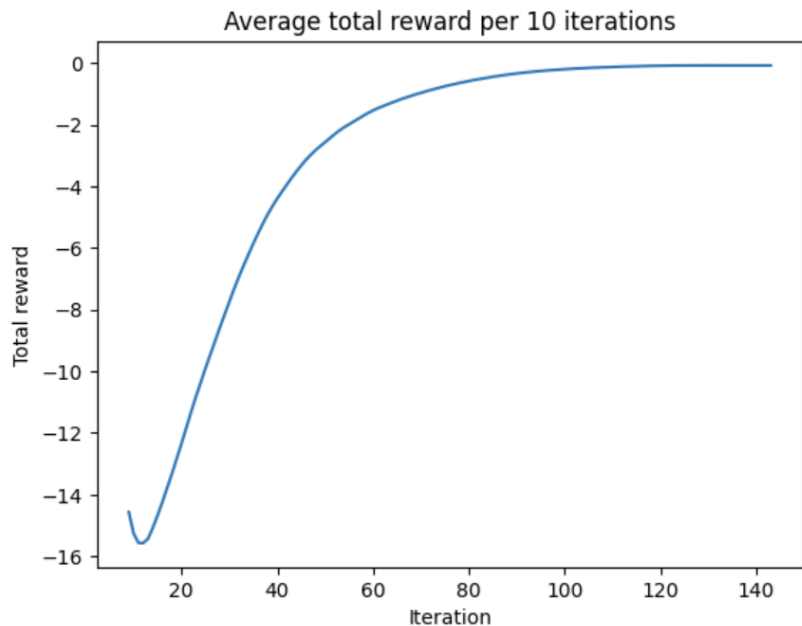
Each episode the model is trained for 8 epochs to avoid overfitting on one sample.

03

Empirical part

Training and validation results

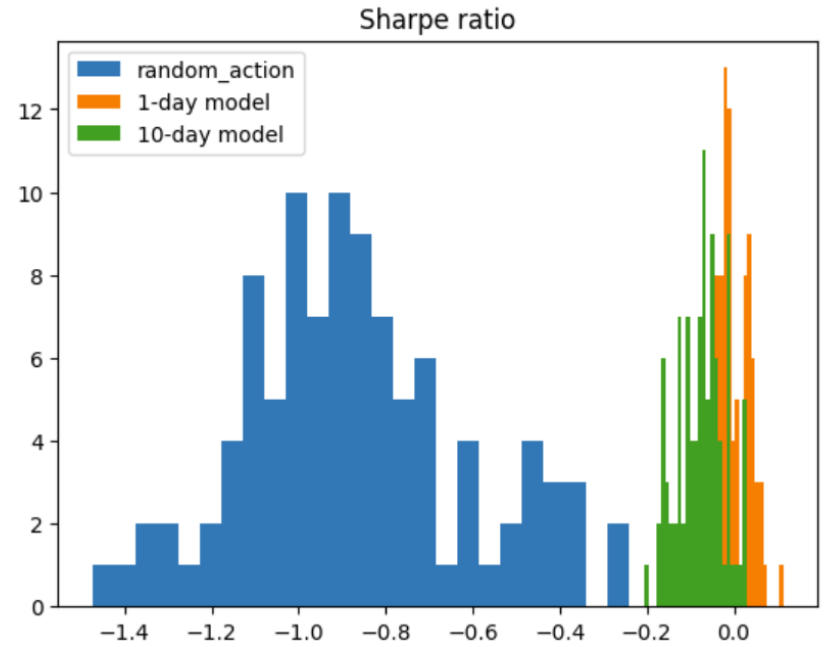
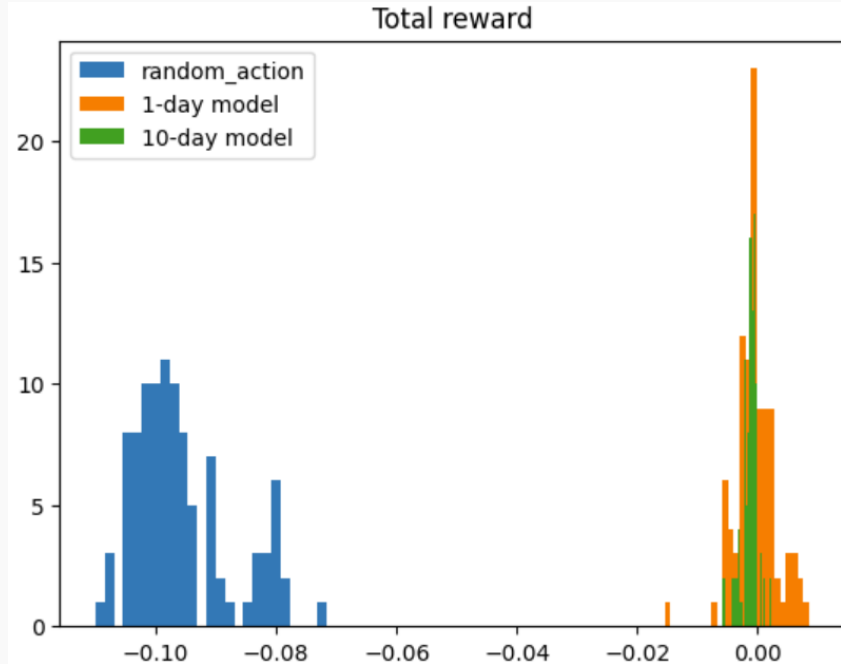
Training on 1-day episodes



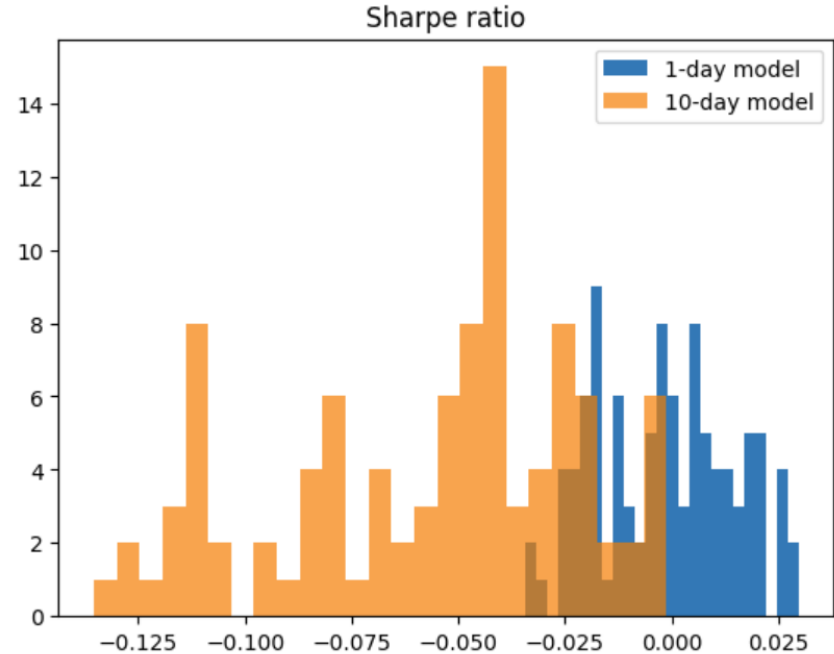
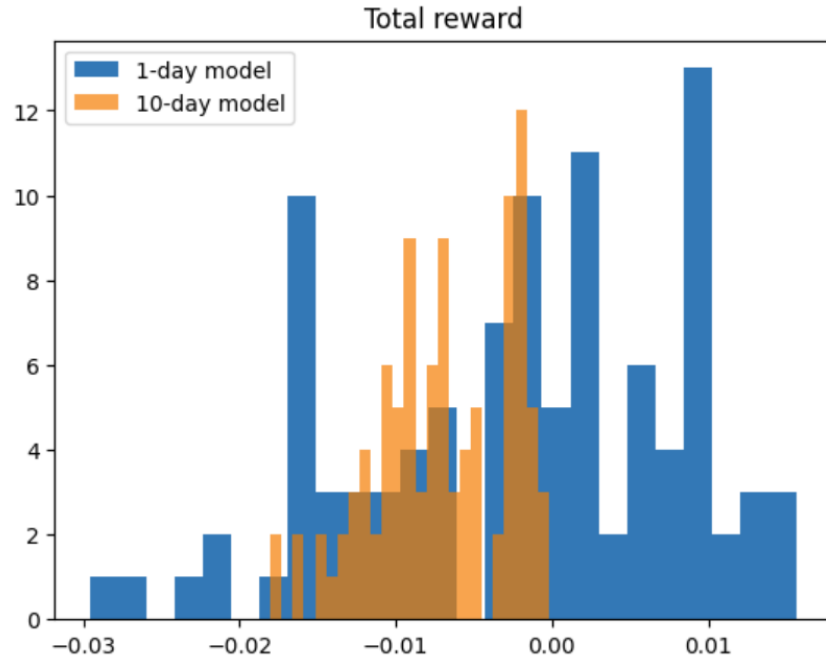
Training on 10-day episodes



Validation set (1-day episodes)

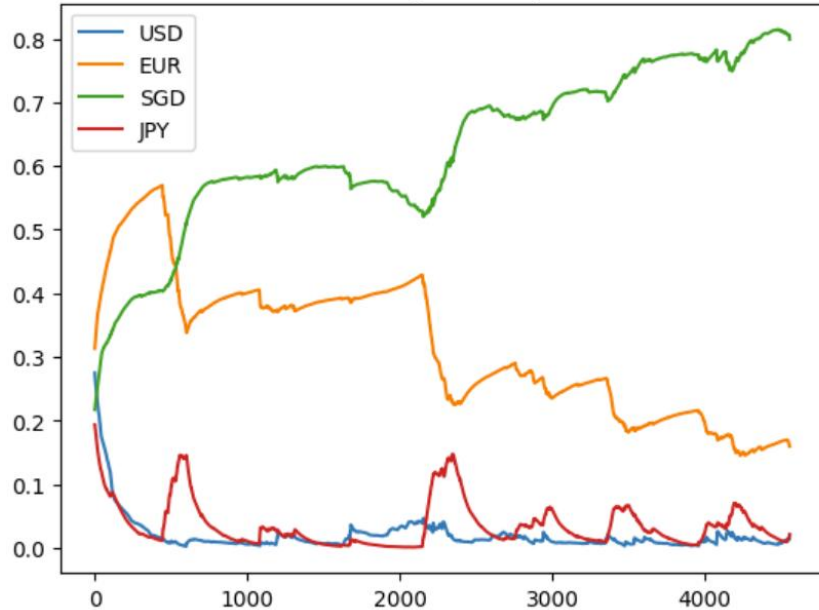


Validation set (10-day episodes)

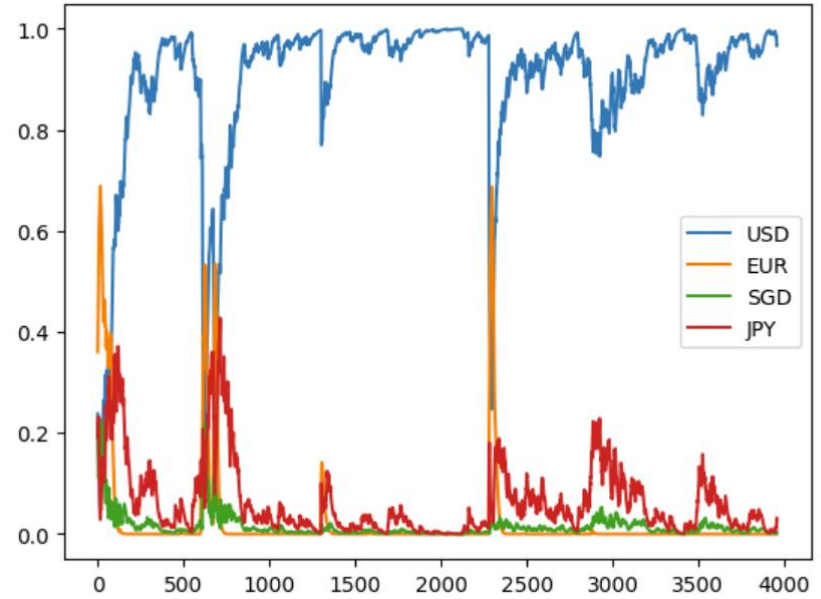


Validation set (10-day episodes)

Portfolio weights 1-day model



Portfolio weights 10-day model



04

Conclusion

Limitations, Further Research

Future research

- Try model-based approaches. For example, DreamerV3 or a custom model based on SDE
 - Add more features:
 - Macroeconomic data (interest rates, inflation)
 - Other relevant factors (sentiment index, futures prices)
 - Train model on longer periods (e.g. 1 month) and lower frequency (e.g. 5-minutes) to allow for higher latency
-

Bibliography

1. Mohammadshafie, A., Mirzaeinia, A., Jumakhan, H., & Mirzaeinia, A. (2024). Deep reinforcement learning strategies in finance: Insights into asset holding, trading behavior, and purchase diversity. In *Proceedings of the CSCE /ICAI'24*. <https://arxiv.org/abs/2407.09557>
 2. Ye, Y., Pei, H., Wang, B., Chen, P.-Y., Zhu, Y., Xiao, J., & Li, B. (2020). Reinforcement-learning based portfolio management with augmented asset movement prediction states. arXiv preprint arXiv:2002.05780. <https://arxiv.org/abs/2002.05780>
 3. Nawathe, S., Panguluri, R., Zhang, J., & Venkatesh, S. (2024). Multimodal deep reinforcement learning for portfolio optimization. arXiv preprint arXiv:2412.17293. <https://arxiv.org/abs/2412.17293>
 4. Lu, L. (2025). Technical Indicator Networks (TINs): An interpretable neural architecture modernizing classical technical analysis for adaptive algorithmic trading. arXiv preprint arXiv:2507.20202. <https://arxiv.org/abs/2507.20202>
-